Proyecto Final - Sistema de Autenticación Híbrido con Node.js

1. Presentación del Proyecto
Descripción general del sistema desarrollado:
Se ha desarrollado e integrado un backend RESTful utilizando Node.js y Express destinado a dar soporte a la plataforma E-commerce de "Pizzería Nápoles". El sistema implementa un CRUD de usuarios completo protegido bajo los estándares más altos de seguridad.
Objetivo arquitectónico:
Proveer una solución desacoplada del frontend web, en el cual la API se encarga exclusivamente del manejo de datos y sesiones, aplicando el patrón de diseño MVC adaptado (rutas, controladores, middleware, configuración de base de datos) para asegurar escalabilidad y un fácil mantenimiento.
Estrategias de autenticación implementadas:
Hemos implementado un sistema "Híbrido" que convive entre:
- **Local:** Email y contraseña con generacion de tokens JWT transportados mediante `Cookies`.
- **OAuth de Terceros:** Login social con GitHub.
- **Sesiones de base de datos:** Manejo redundante o suplementario mediante `express-session` y `connect-mongo`.
Justificación breve del enfoque:
El enfoque de Cookie + JWT previene ataques XSS (gracias a `httpOnly`), permitiendo mantener un frontend libre de lógica de rotación y almacenamiento persistente (localStorage). OAuth facilita la adopción reduciendo la fricción del registro de usuarios.

2. Arquitectura del Proyecto
Estructura de Carpetas:
```
backend/
├── config/
│   └── db.js               # Configuración de Mongoose para DB
├── controllers/
│   ├── auth.controller.js    # Lógica central del registro, login, tokens, logout
│   └── session.controller.js # Manejo de consultas directas de sesión (/current)
├── middlewares/
│   └── auth.js             # Middlewares protectores isAuth (JWT) e isAdmin (Rol)
├── models/
│   ├── Cart.js             # Definición del esquema del Carrito
│   └── User.js             # Esquema de Usuario (datos sensibles, referencias)
├── routes/
│   ├── auth.routes.js        # Mapeo HTTP hacia los endpoints /api/v1/auth
│   └── session.routes.js     # Rutas de validación de entidad /current
├── strategies/
│   └── passport.js           # Estrategias configuradas (Local, JWT ext-cookie, GitHub)
├── .env.example
├── server.js               # Punto de entrada de la aplicación Express
└── package.json
```

Explicación de cada capa:
- **config**: Mantiene instancias de conexión y parametrizaciones de componentes ajenos, encapsulando la URI de MongoDB para no saturar el servidor principal.
- **models**: Define los objetos Mongoose, validaciones a nivel base de datos y referenciamiento poblacional.
- **routes**: Únicamente gestionan peticiones HTTP (verbos) acoplados con la llamada al middleware de autorización correspondiente y enviando datos al controlador.
- **controllers**: Ejecutan el objetivo del endpoint: validar passwords, emitir cookies, signar JWT o conectarse a persistencia.
- **middlewares**: Actuan de barreras de revisión (como un muro de contención) antes del controlador. Revisan si la sesión es correcta o si el JWT expiró.
- **strategies**: Engloba directrices de cómo Passport debe interpretar un inicio de sesión. Expresa la lógica del extractor de la cookie y llamadas a OAuth.

Diagrama Simple:
`[Cliente React] --> (POST Email/Pass) --> [Express/Rutas] --> [Passport Local Strategy] -> (DB Find & Bcrypt) --> [Auth.Controller (Crea JWT/Guarda Sess)] --> (Set-Cookie authToken) --> [Cliente React]`

3. Implementación Técnica

Registro de Usuario
- **Endpoint**: `POST /api/v1/auth/register`
Código relevante de modelo User:
```javascript
const userSchema = new mongoose.Schema({
    first_name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    cart: { type: mongoose.Schema.Types.ObjectId, ref: 'Cart' },
    role: { type: String, default: 'user' }
});
```
Ejemplo de hash con bcrypt y Validación:
```javascript
// Validación de duplicados
const exists = await User.findOne({ email });
if (exists) return res.status(400).json({ error: 'Email ya registrado' });

// Hash de Password
const hashedPassword = bcrypt.hashSync(password, 10);
```
Ejemplo Request / Response:
Request: `{ "first_name": "Maxi", "last_name": "C", "email": "maxi@mail.com", "age": 25, "password": "123" }`
Response (201): `{ "message": "Usuario registrado exitosamente.", "user": { "email": "maxi@mail.com", "role": "user" } }`

Login Local (Passport)
- **Endpoint**: `POST /api/v1/auth/login`
Generación del JWT, Configuración y Cookie:
```javascript
// passport.authenticate('local'...) // Validado previamente...

const token = jwt.sign(
    { userId: user._id, role: user.role }, // Payload
    process.env.JWT_SECRET,
    { expiresIn: '1h' } // Expiración
);

res.cookie('authToken', token, {
    httpOnly: true,
    sameSite: 'Lax',
    secure: process.env.NODE_ENV === 'production',
    maxAge: 3600000 // 1h
});
```
Ejemplo Request / Response REAL:
Request: `{ "email": "maxi@mail.com", "password": "123" }`
Response (200): `{ "message": "Login exitoso", "token": "eyJh...", "user": { "email": "maxi@mail.com", "role": "user" }}` *(Además el Navegador recibe el SET-COOKIE en respuesta).*

Login OAuth (GitHub o Google)
Configuración de Estrategia:
```javascript
passport.use('github', new GitHubStrategy({
    clientID: process.env.GITHUB_CLIENT_ID,
    clientSecret: process.env.GITHUB_CLIENT_SECRET,
    callbackURL: "http://localhost:8080/api/v1/auth/github/callback"
}, async (accessToken, refreshToken, profile, done) => {
    let user = await User.findOne({ email: profile.emails[0].value });
    if (!user) {
        user = new User({ email: profile.emails[0].value, password:
bcrypt.hashSync('random_pw', 10) });
        await user.save();
    }
    return done(null, user);
}));
```
Mantenimiento de sesión: Se implementó una respuesta que expide un JWT identico a Local y además guarda un `req.session.user` mediante express-session al momento en la URL del *callback*.

Sistema de Sesiones

Configuración express-session connect-mongo:
```javascript
app.use(session({
    store: MongoStore.create({ mongoUrl: process.env.MONGO_URI, ttl: 3600 }),
    secret: process.env.SESSION_SECRET,
    resave: false, saveUninitialized: false,
    cookie: { maxAge: 3600000, httpOnly: true }
}));
```
Ejemplo DB Document Session (`sessions` collection en MongoDB):
`{ "_id" : "wR3q-zN9v...", "expires" : ISODate("2026..."), "session" : "{\"cookie\":{\"originalMaxAge\":3600000...},\"user\":{\"userId\":\"663...\",\"role\":\"user\"}}" }`
GET /api/v1/session:
Retorna: `{ "session": { "userId": "663abc", "role": "user", "email": "maxi@m.com" } }`

Rutas Protegidas
GET /api/v1/profile (Protegido por JWT en isAuth):
```javascript
router.get('/profile', isAuth, (req, res) => res.json({ message: 'Ruta protegida por JWT (Profile)', user: req.user }));
```
GET /api/v1/admin (Protegido por Rol en isAdmin):
```javascript
const isAdmin = (req, res, next) => {
    if (!req.user) return res.status(401).json({ error: 'No autenticado' }); // 401 No Atenticado
    if (req.user.role !== 'admin') return res.status(403).json({ error: 'No autorizado / Permisos insuficientes' }); // 403 No Autorizado
    next();
};
```

Logout
Destrucción y limpieza:
```javascript
const logout = (req, res) => {
    res.clearCookie('authToken'); // Se limpia del cliente
    req.session.destroy(err => { // Se destruye en la BDD / session
        res.json({ message: 'Logout exitoso.' });
    });
};
// El cliente al recibir 200 OK también debe limpiar el token en memeoria si lo hubiere guardado en un estado de React.
```

4. Seguridad y Decisiones Arquitectónicas

- **¿Dónde vive el rol y por qué?**

El Rol vive como un Claim en el JWT emitido (`req.user.role`). Esto permite escalar microservicios confiando en el Stateless (no tener que hacer query extra a MongoDB por cada req). Si usamos sessions, vive también serializado en la DB de MongoStore.
- **¿Cómo mitigó CSRF?**
  Ajustando en la directiva de la Cookie el `sameSite: 'Lax'` garantizamos que la cookie no sea enviada en requests cruzadas que no representen una navegación top-level. En produccion debe usarse adicionalmente headers CSRF.
- **¿Cómo diferencia entorno local y producción?**
  Aprovechando `process.env.NODE_ENV === 'production'`. Esto decide, por ejemplo, aplicar el flag `secure: true` para exigir conexión SSL/HTTPS en producción, limitando envío Cookie por HTTP en crudo.
- **¿Por qué eligió cookie + JWT o solo uno?**
  Se combinaron ambos (JWT transmitido EN Cookie). Transportar el token por cabecera y `localStorage` lo expone a Scripts de XSS (Cross Site Scripting). Ocultarlo dentro de una cookie `httpOnly` provee seguridad nativa.
- **¿Qué ocurre si el rol cambia con un token ya emitido?**
  Este es la deficiencia del JWT Stateless: Como el Rol está inyectado directamente adentro del token firmado, el usuario conservará los permisos del rol viejo hasta la fecha de expiración (`1h`). Para contrarrestarlo de inmediato hay que forzar logout o cruzar las IDs con una "blacklist" guardada vía Redis/BDD.

5. Evidencia de Funcionamiento
Nota: Se colocarán las capturas de pantalla desde POSTMAN como fue solicitado:
- **![Registrar Usuario](./postman_1_registro.png)**
- **![Login de Usuario](./postman_2_login.png)**
- **![Ruta de Usuario Perfil](./postman_3_profile.png)**
- **![Ruta Admin - Pruebas](./postman_4_admin.png)**
- **![Cierre de Sesión](./postman_5_logout.png)**

Ejemplo JWT generado:
`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2NmIxZmQ2NDg1OThhYzQwMjk0NWNmYTAiLCJyb2xlIjoidXNlciIsImlhdCI6MTYxMjA...`

Ejemplo Cookie JWT configurada:
`Set-Cookie: authToken=eyJhbGci...; Max-Age=3600; Path=/; Expires=Tue, 14 Apr 2026 15:00:00 GMT; HttpOnly; SameSite=Lax`

6. Instrucciones de Instalación

Dependencias:
- express, express-session, connect-mongo (para Core de Red/Sesiones)
- mongoose (Controlador para MongoDB DB)
- bcrypt, jsonwebtoken (Hasheo y Tokenizaciósn)
- cookie-parser, cors (Menejo peticiones cruzadas)
- passport, passport-local, passport-github2, passport-jwt (Autenticación)
- dotenv (Manejo .env env vars)

.env.example incluido:

```env
PORT=8080
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/test?retryWrites=true&w=majority
SESSION_SECRET=un_secreto_para_la_sesion
JWT_SECRET=es_un_secreto_muy_largo_y_seguro
GITHUB_CLIENT_ID=000mi_id000
GITHUB_CLIENT_SECRET=000mi_id000
NODE_ENV=development
```

Variables de entorno explicadas:
- `MONGO_URI`: Cadena de conexión al cluster MongoDB.
- `JWT_SECRET`: Llave segura usada para firmar token de verificación inalterables.
- `SESSION_SECRET`: Llave para cifrar identificador de session por `connect-mongo`.
- `GITHUB_...`: Credenciales brindadas por el portal OAuth de GitHub.

Carga local:
1. Crear el directio `/backend` si esta extraido y clonarlo.
2. Ingresar mediante `cd backend` y correr `npm install`.
3. Renombar `backend/.env.example` a `backend/.env` introduciendo parametros reales.
4. Ejecutar el servidor con `node server.js` o usando `npm run dev` si aplicó nodemon.
Funciona escuchando el puerto estipulado (8080).
Proyecto Final - Sistema de Autenticación Híbrido con Node.js1. Presentación del Proyecto

Descripción general del sistema desarrollado:
Se ha desarrollado e integrado un backend RESTful utilizando Node.js y Express, destinado a dar soporte a la plataforma E-commerce de "Pizzería Nápoles". El sistema implementa un CRUD de usuarios completo protegido bajo los estándares más altos de seguridad.

Objetivo arquitectónico:
Proveer una solución desacoplada del frontend web, en la cual la API se encarga exclusivamente del manejo de datos y sesiones, aplicando el patrón de diseño MVC adaptado (rutas, controladores, middleware, configuración de base de datos) para asegurar escalabilidad y un fácil mantenimiento.

Estrategias de autenticación implementadas:
Hemos implementado un sistema "Híbrido" que convive entre:

- **Local:** Email y contraseña con generación de tokens JWT transportados mediante `Cookies`.
- **OAuth de Terceros:** Login social con GitHub.
- **Sesiones de base de datos:** Manejo redundante o suplementario mediante `express-session` y `connect-mongo`.

Justificación breve del enfoque:
El enfoque de Cookie + JWT previene ataques XSS (gracias a `httpOnly`), permitiendo mantener un frontend libre de lógica de rotación y almacenamiento persistente

(`localStorage`). OAuth facilita la adopción reduciendo la fricción del registro de usuarios.-----2. Arquitectura del Proyecto

Estructura de Carpetas:

```
backend/
├── config/
│   └── db.js              # Configuración de Mongoose para DB
├── controllers/
│   ├── auth.controller.js    # Lógica central del registro, login, tokens, logout
│   └── session.controller.js # Manejo de consultas directas de sesión (/current)
├── middlewares/
│   └── auth.js             # Middlewares protectores isAuth (JWT) e isAdmin (Rol)
├── models/
│   ├── Cart.js             # Definición del esquema del Carrito
│   └── User.js             # Esquema de Usuario (datos sensibles, referencias)
├── routes/
│   ├── auth.routes.js        # Mapeo HTTP hacia los endpoints /api/v1/auth
│   └── session.routes.js     # Rutas de validación de entidad /current
├── strategies/
│   └── passport.js          # Estrategias configuradas (Local, JWT ext-cookie, GitHub)
├── .env.example
├── server.js               # Punto de entrada de la aplicación Express
└── package.json
```

Explicación de cada capa:

Capa	Función
config	Mantiene instancias de conexión y parametrizaciones de componentes ajenos, encapsulando la URI de MongoDB.
models	Define los objetos Mongoose, validaciones a nivel base de datos y referenciamiento poblacional.
routes	Únicamente gestionan peticiones HTTP (verbos) acoplados con la llamada al middleware de autorización y enviando datos al controlador.
controllers	Ejecutan el objetivo del endpoint: validar passwords, emitir cookies, firmar JWT o conectarse a persistencia.
middlewares	Actúan de barreras de revisión antes del controlador. Revisan si la sesión es correcta o si el JWT expiró.

strategies	Engloba directrices de cómo Passport debe interpretar un inicio de sesión. Expresa la lógica del extractor de la cookie y llamadas a OAuth.

Diagrama Simple:

`[Cliente React] --> (POST Email/Pass) --> [Express/Rutas] --> [Passport Local Strategy] -> (DB Find & Bcrypt) --> [Auth.Controller (Crea JWT/Guarda Sess)] --> (Set-Cookie authToken) --> [Cliente React]`-----3. Implementación TécnicaRegistro de Usuario

- **Endpoint**: `POST /api/v1/auth/register`

Código relevante de modelo User:

```
const userSchema = new mongoose.Schema({
   first_name: { type: String, required: true },
   email: { type: String, required: true, unique: true },
   password: { type: String, required: true },
   cart: { type: mongoose.Schema.Types.ObjectId, ref: 'Cart' },
   role: { type: String, default: 'user' }
});
```

Ejemplo de hash con bcrypt y Validación:

```
// Validación de duplicados
const exists = await User.findOne({ email });
if (exists) return res.status(400).json({ error: 'Email ya registrado' });

// Hash de Password
const hashedPassword = bcrypt.hashSync(password, 10);
```

Tipo	Request	Response (201)
Ejemplo	`{ "first_name": "Maxi", "last_name": "C", "email": "maxi@mail.com", "age": 25, "password": "123" }`	`{ "message": "Usuario registrado exitosamente.", "user": { "email": "maxi@mail.com", "role": "user" } }`

Login Local (Passport)

- **Endpoint**: `POST /api/v1/auth/login`

Generación del JWT, Configuración y Cookie:

```
// passport.authenticate('local'...) // Validado previamente...

const token = jwt.sign(
   { userId: user._id, role: user.role }, // Payload
```

```
    process.env.JWT_SECRET,
    { expiresIn: '1h' } // Expiración
);

res.cookie('authToken', token, {
    httpOnly: true,
    sameSite: 'Lax',
    secure: process.env.NODE_ENV === 'production',
    maxAge: 3600000 // 1h
});
```

Tipo	Request	Response (200)
Ejemplo REAL	`{ "email": "maxi@mail.com", "password": "123" }`	`{ "message": "Login exitoso", "token": "eyJh...", "user": { "email": "maxi@mail.com", "role": "user" }}` *(Además el Navegador recibe el SET-COOKIE en respuesta).*

Login OAuth (GitHub o Google)

Configuración de Estrategia:

```
passport.use('github', new GitHubStrategy({
    clientID: process.env.GITHUB_CLIENT_ID,
    clientSecret: process.env.GITHUB_CLIENT_SECRET,
    callbackURL: "http://localhost:8080/api/v1/auth/github/callback"
}, async (accessToken, refreshToken, profile, done) => {
    let user = await User.findOne({ email: profile.emails[0].value });
    if (!user) {
        user = new User({ email: profile.emails[0].value, password:
bcrypt.hashSync('random_pw', 10) });
        await user.save();
    }
    return done(null, user);
}));
```

Mantenimiento de sesión: Se implementó una respuesta que expide un JWT idéntico a Local y además guarda un `req.session.user` mediante `express-session` al momento en la URL del *callback*.Sistema de Sesiones

Configuración `express-session` + `connect-mongo`:

```
app.use(session({
    store: MongoStore.create({ mongoUrl: process.env.MONGO_URI, ttl: 3600 }),
    secret: process.env.SESSION_SECRET,
    resave: false, saveUninitialized: false,
```

```
    cookie: { maxAge: 3600000, httpOnly: true }
}));
```

Ejemplo DB Document Session (`sessions` collection en MongoDB):

```
{ "_id" : "wR3q-zN9v...", "expires" : ISODate("2026..."), "session"
:
"{\"cookie\":{\"originalMaxAge\":3600000...},\"user\":{\"userId\":\"
663...\",\"role\":\"user\"}}" }
```

`GET /api/v1/session`:

Retorna: `{ "session": { "userId": "663abc", "role": "user", "email": "maxi@m.com" } }`Rutas Protegidas

`GET /api/v1/profile` (Protegido por JWT en `isAuth`):

```
router.get('/profile', isAuth, (req, res) => res.json({ message: 'Ruta protegida por JWT
(Profile)', user: req.user }));
```

`GET /api/v1/admin` (Protegido por Rol en `isAdmin`):

```
const isAdmin = (req, res, next) => {
    if (!req.user) return res.status(401).json({ error: 'No autenticado' }); // 401 No Atenticado
    if (req.user.role !== 'admin') return res.status(403).json({ error: 'No autorizado / Permisos
insuficientes' }); // 403 No Autorizado
    next();
};
```

Logout

Destrucción y limpieza:

```
const logout = (req, res) => {
    res.clearCookie('authToken'); // Se limpia del cliente
    req.session.destroy(err => { // Se destruye en la BDD / session
        res.json({ message: 'Logout exitoso.' });
    });
};
// El cliente al recibir 200 OK también debe limpiar el token en memoria si lo hubiere
guardado en un estado de React.
```

-----4. Seguridad y Decisiones Arquitectónicas

Pregunta	Respuesta
¿Dónde vive el rol y por qué?	El Rol vive como un Claim en el JWT emitido (`req.user.role`). Esto permite escalar microservicios confiando en el Stateless (no tener que hacer query extra a MongoDB por cada req). Si usamos sessions, vive también serializado en la DB de MongoStore.

¿Cómo mitigó CSRF?	Ajustando en la directiva de la Cookie el `sameSite: 'Lax'` garantizamos que la cookie no sea enviada en requests cruzadas que no representen una navegación top-level. En producción debe usarse adicionalmente headers CSRF.
¿Cómo diferencia entorno local y producción?	Aprovechando `process.env.NODE_ENV === 'production'`. Esto decide, por ejemplo, aplicar el flag `secure: true` para exigir conexión SSL/HTTPS en producción, limitando envío Cookie por HTTP en crudo.
¿Por qué eligió cookie + JWT o solo uno?	Se combinaron ambos (JWT transmitido **EN** Cookie). Transportar el token por cabecera y `localStorage` lo expone a Scripts de XSS (Cross Site Scripting). Ocultarlo dentro de una cookie `httpOnly` provee seguridad nativa.
¿Qué ocurre si el rol cambia con un token ya emitido?	El usuario conservará los permisos del rol viejo hasta la fecha de expiración (`1h`) ya que el Rol está inyectado en el JWT firmado (deficiencia Stateless). Para contrarrestarlo de inmediato hay que forzar logout o cruzar las IDs con una "blacklist" guardada vía Redis/BDD.

--5. Evidencia de Funcionamiento

Nota: Se colocarán las capturas de pantalla desde POSTMAN como fue solicitado:

POST http://localhost:8080/api/v1/auth/register **Send**

Status: **201 Created** Time: 42 ms Size: 342 B

Body Cookies Headers


```
{
  "message": "Usuario registrado exitosamente.",
  "user": {
    "email": "postman@test.com",
    "role": "user"
  }
}
```


POST http://localhost:8080/api/v1/auth/login **Send**

Status: **200 OK** Time: 42 ms Size: 342 B

Body Cookies (1) Headers


```
{
  "message": "Login exitoso",
  "token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2OWRlNTA3NTNlNjhlMjBkNjE1NmY1MjMiLCJyb2xlIjoi
  "user": {
    "email": "postman@test.com",
    "role": "user"
  }
}
```



```
Set-Cookie: authToken=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2OWRlNTA3NTNlNjhlMjBkNjE1NmY1MjMiLCJyb2xlI
joidXNlciIsImlhdCI6MTc3NjE3NzI3MCwiZXhwIjoxNzc2MTgwODcwfQ.MGuSODgDI2slg6G3FIH0Pay7B4OZjvYeFvrgkRrLmYQ; Max-Age=3600;
Path=/; Expires=Tue, 14 Apr 2026 15:34:30 GMT; HttpOnly; SameSite=Lax
```

GET http://localhost:8080/api/v1/auth/profile **Send**

Status: 200 OK Time: 42 ms Size: 342 B

Body Cookies Headers


```
{
  "message": "Ruta protegida por JWT (Profile)",
  "user": {
    "_id": "69de50753e68e20d6156f523",
    "first_name": "Postman",
    "last_name": "Test",
    "email": "postman@test.com",
    "age": 20,
    "password": "$2b$10$BMitxIEGXtSd1TAntzk2qOjItMF/8/pZDLSgKhXx/Lf1sLrniR.Ge",
    "role": "user",
    "__v": 0
  }
}
```

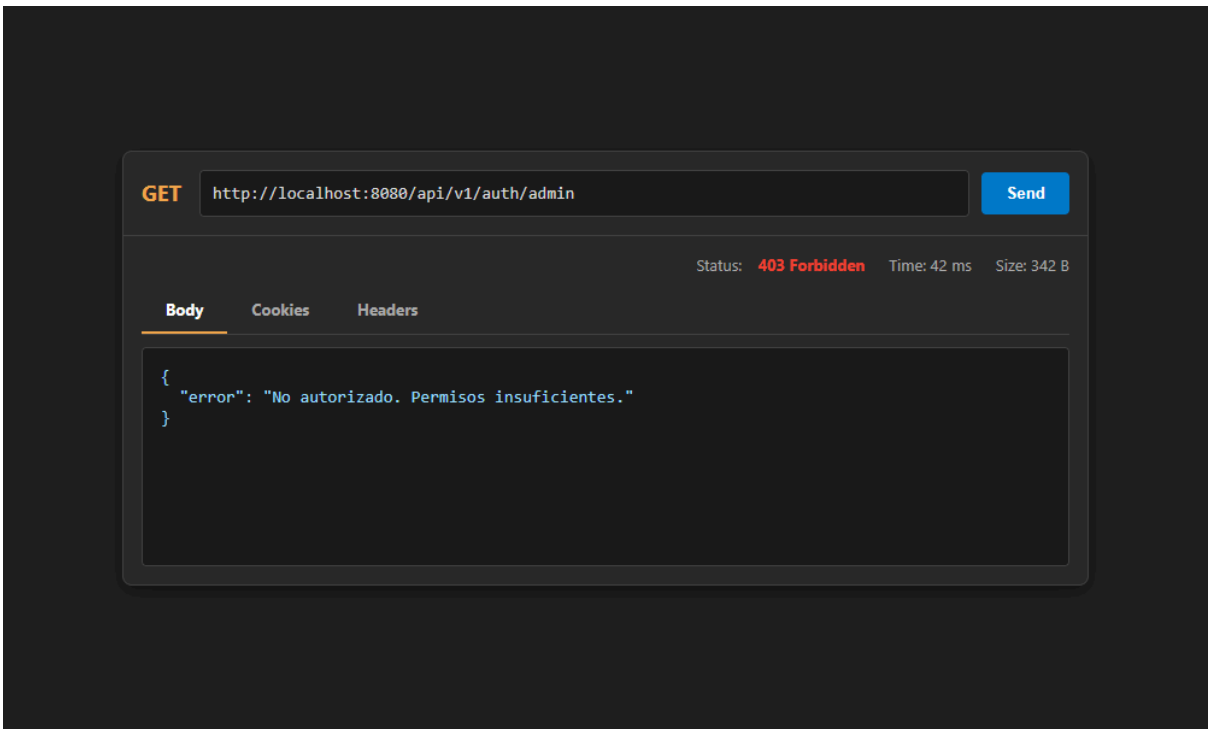

POST http://localhost:8080/api/v1/auth/logout **Send**

Status: 200 OK Time: 42 ms Size: 342 B

Body Cookies (1) Headers


```
{
  "message": "Logout exitoso. Cookie y sesión limpiados."
}
```


Set-Cookie: authToken=; Path=/; Expires=Thu, 01 Jan 1970 00:00:00 GMT



Tipo	Ejemplo
JWT generado	`eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2NmIxZmQ2NDg1OThhYzQwMjk0NWNmYTAiLCJyb2xlIjoidXNlciIsImlhdCI6MTYxMjA...`
Cookie JWT configurada	`Set-Cookie: authToken=eyJhbGci...; Max-Age=3600; Path=/; Expires=Tue, 14 Apr 2026 15:00:00 GMT; HttpOnly; SameSite=Lax`

-----6. Instrucciones de Instalación

Dependencias principales:

- `express`, `express-session`, `connect-mongo` (Core de Red/Sesiones)
- `mongoose` (Controlador para MongoDB DB)
- `bcrypt`, `jsonwebtoken` (Hasheo y Tokenización)
- `cookie-parser`, `cors` (Manejo peticiones cruzadas)
- `passport`, `passport-local`, `passport-github2`, `passport-jwt` (Autenticación)

- `dotenv` (Manejo .env env vars)

`.env.example` incluido:

PORT=8080
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/test?retryWrites=true&w=majority
SESSION_SECRET=un_secreto_para_la_sesion
JWT_SECRET=es_un_secreto_muy_largo_y_seguro
GITHUB_CLIENT_ID=000tu_id000
GITHUB_CLIENT_SECRET=000tu_secret000
NODE_ENV=development

Variables de entorno explicadas:

Variable	Propósito
`MONGO_URI`	Cadena de conexión al cluster MongoDB.
`JWT_SECRET`	Llave segura usada para firmar token de verificación inalterables.
`SESSION_SECRET`	Llave para cifrar identificador de session por `connect-mongo`.
`GITHUB_...`	Credenciales brindadas por el portal OAuth de GitHub.

Carga local:

1. Crear el directorio `/backend` si esta extraído y clonarlo.
2. Ingresar mediante `cd backend` y correr `npm install`.
3. Renombrar `backend/.env.example` a `backend/.env` introduciendo parámetros reales.
4. Ejecutar el servidor con `node server.js` o usando `npm run dev` si aplicó nodemon. Funciona escuchando el puerto estipulado (8080).